



SDI – Sistemas Distribuidos e Internet

ENUNCIADO PRÁCTICA 1 – SPRING

INFORME

Nombre/Apellidos, email de UNIOV, I ID-GIT#1	Omayma El Imami Gaada , UO300838@uniovi.es , 2526-703
Nombre/Apellidos, email de UNIOV, ID-GIT#2	Ikram El Mabrouk Morhnane , UO301848@uniovi.es , 2526-704
Nombre/Apellidos, email de UNIOV, ID-GIT#3	Moisés Montes Iglesias, UO296102@uniovi.es ,2526-710
Nombre/Apellidos, email de UNIOV, ID-GIT#4	Mario de la Mora Martínez , UO281860@uniovi.es ,2526-711
Cód. ID EQUIPO	72
URL Repositorio GitHub	https://github.com/MoisesMontesIglesias/sdi2526-entrega1-72

Índice



1	INTRODUCCIÓN	2
2	DIAGRAMA DE NAVEGABILIDAD.....	2
3	DIAGRAMA ENTIDAD-RELACIÓN.....	3
4	ASPECTOS TÉCNICOS Y DE DISEÑO RELEVANTES.....	3
5	DESCRIPCIÓN DE FUNCIONALIDADES DESARROLLAS POR CADA MIEMBRO DEL EQUIPO	3
6	CONCLUSIÓN INDIVIDUAL DE CADA MIEMBRO DEL GRUPO.....	3
7	INFORMACIÓN NECESARIA PARA EL DESPLIEGUE Y EJECUCIÓN	3
8	CONCLUSIÓN	3

1 Introducción

El objetivo de esta práctica es el desarrollo de una aplicación web para la gestión de reservas de espacios dentro de una organización. La aplicación permite a los usuarios reservar recursos como salas de reuniones, aulas o puestos de coworking, facilitando la planificación y organización de estos espacios compartidos.



El sistema ha sido desarrollado utilizando el framework **Spring Boot**, siguiendo una arquitectura basada en capas que separa claramente la lógica de negocio, el acceso a datos y la presentación. Este enfoque permite mantener un código modular, escalable y fácil de mantener.

La aplicación distingue entre dos tipos de usuarios: **usuarios estándar** y **administradores**. Los usuarios estándar pueden consultar espacios disponibles, verificar su disponibilidad, realizar reservas y gestionar sus propias reservas. Por otro lado, los administradores disponen de funcionalidades adicionales que permiten gestionar los espacios del sistema, crear bloqueos de mantenimiento, consultar el listado global de reservas, así como exportar el mismo.

Además, el sistema implementa mecanismos de **seguridad y control de acceso mediante Spring Security**, garantizando que cada usuario solo pueda acceder a los recursos correspondientes a su rol. También se han incorporado validaciones para evitar inconsistencias en los datos, como reservas solapadas o accesos no autorizados.

En este documento se describen los principales aspectos del diseño y desarrollo de la aplicación, incluyendo el modelo de datos, el diagrama de navegabilidad, las decisiones técnicas adoptadas y la distribución del trabajo entre los miembros del equipo.

2 Diagrama de navegabilidad

El diagrama de navegabilidad representa el flujo de interacción entre las distintas vistas de la aplicación y muestra cómo los usuarios pueden desplazarse entre ellas en función de su rol y de las acciones que realizan dentro del sistema.

La aplicación comienza en la zona pública, donde el usuario puede acceder a la página principal. La ruta / muestra la vista home cuando no existe una sesión activa y desde ella el usuario puede iniciar sesión o registrarse en el sistema. En caso de que no disponga de cuenta, puede completar el proceso de registro mediante el formulario correspondiente. Si un usuario intenta acceder a una ruta privada sin haber iniciado sesión, el sistema lo redirige automáticamente a la página de login.

Una vez que el usuario se autentica —ya sea tras iniciar sesión o completar el registro— el sistema lo redirige automáticamente en función de su rol: los usuarios con rol USER acceden a **/spaces**, mientras que los usuarios con rol ADMIN son dirigidos a **/reservas/listado-global**.

Los usuarios con rol USER acceden al listado de espacios disponibles, accesible mediante las rutas **/spaces** y **/espacios/disponibles**, desde el cual pueden consultar los espacios activos de la organización. A partir de esta vista es posible navegar al detalle de cada espacio para consultar su información y disponibilidad. Desde el detalle se puede iniciar el proceso de creación de una reserva, que incluye la validación de disponibilidad para evitar solapamientos con otras reservas o con bloqueos de mantenimiento. Este proceso contempla además funcionalidades como la gestión de reservas recurrentes y la limitación del número de reservas activas por usuario, aunque estas no se exponen como endpoints independientes.

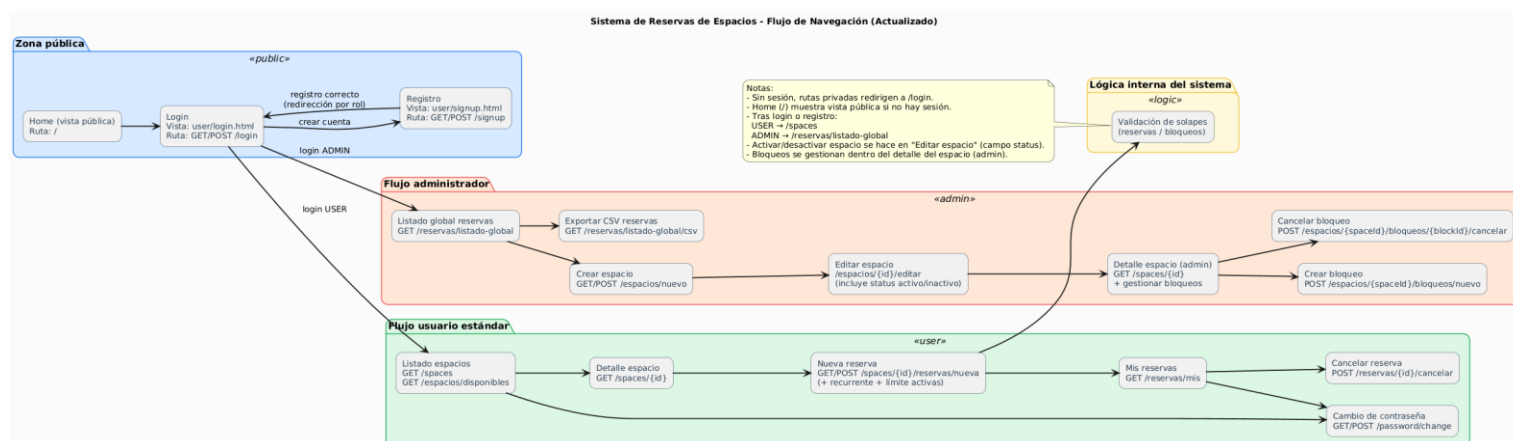
Los usuarios estándar también disponen de una sección de “mis reservas”, donde pueden consultar todas las reservas asociadas a su cuenta, tanto activas como canceladas. Desde esta vista pueden cancelar reservas existentes y acceder a funcionalidades propias del usuario estándar, como el cambio de contraseña.

Por otro lado, los usuarios con rol ADMIN acceden al listado global de reservas del sistema. Esta sección permite consultar todas las reservas registradas y realizar tareas de gestión relacionadas con los espacios

disponibles. Desde este panel es posible crear nuevos espacios y editar su información, incluyendo su estado (activo o inactivo). Además, desde el detalle de cada espacio, el administrador puede gestionar los bloqueos de mantenimiento, creando nuevos bloqueos o cancelando los existentes.

La gestión de bloqueos permite impedir temporalmente la realización de reservas sobre un espacio durante determinados periodos de tiempo, por ejemplo, debido a tareas de mantenimiento o incidencias. Además, el administrador puede exportar el listado global de reservas en formato CSV mediante el endpoint `/reservas/listado-global/csv` para su análisis o tratamiento externo.

En conjunto, el diagrama muestra de forma estructurada cómo se organizan las distintas rutas de la aplicación y cómo la navegación se adapta al rol del usuario, diferenciando claramente las funcionalidades disponibles para usuarios estándar y administradores.



3 Diagrama Entidad-Relación

El modelo de datos de la aplicación se basa en varias entidades principales que representan los elementos fundamentales del sistema de reservas. Estas entidades reflejan los usuarios del sistema, los espacios disponibles para reservar, las reservas realizadas y los posibles bloqueos de mantenimiento que afectan a la disponibilidad de los espacios.

La entidad **User** almacena la información de los usuarios registrados en el sistema. Entre sus atributos se encuentran el DNI, el nombre, los apellidos, la contraseña cifrada y el rol del usuario. El rol permite distinguir entre usuarios estándar (USER) y administradores (ADMIN), determinando así los permisos y funcionalidades disponibles dentro de la aplicación. Además, la entidad incluye el campo **passwordConfirm**, utilizado únicamente durante la validación del formulario de registro y marcado como **@Transient**, por lo que no se almacena en la base de datos.

La entidad **Space** representa los espacios que pueden ser reservados dentro de la organización, como salas de reuniones, aulas o puestos de coworking. Cada espacio incluye información como su nombre, tipo, ubicación, capacidad, una descripción opcional y un estado (**status**) que determina si el espacio está activo (**ACTIVE**) o cancelado (**CANCELLED**). Asimismo, los espacios no se eliminan físicamente de la base

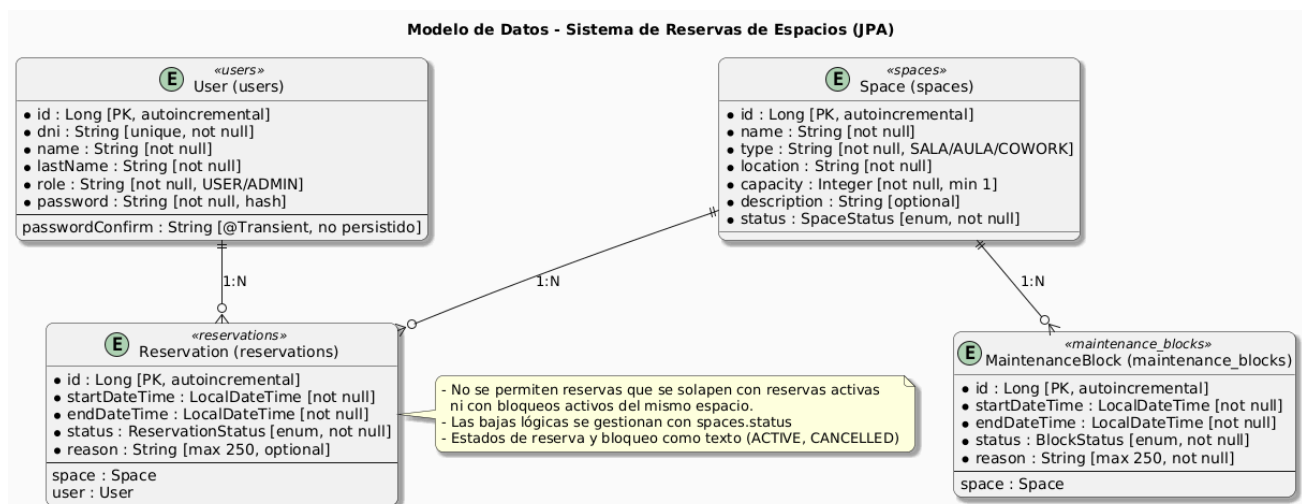
de datos; en su lugar, se gestionan mediante un mecanismo de baja lógica utilizando el campo **status**, lo que permite conservar el historial de reservas asociado a dichos espacios.

La entidad **Reservation** representa las reservas realizadas por los usuarios sobre los distintos espacios. Cada reserva está asociada a un usuario y a un espacio concreto mediante relaciones con las entidades User y Space. Además, incluye información sobre la fecha y hora de inicio (**startDateTime**), la fecha y hora de fin (**endDateTime**), el motivo de la reserva y su estado. El estado se representa mediante el enumerado **ReservationStatus**, que permite distinguir entre reservas activas y canceladas.

Por otro lado, la entidad **MaintenanceBlock** permite registrar periodos en los que un espacio no estará disponible para reservas debido a tareas de mantenimiento u otros motivos. Cada bloqueo está asociado a un espacio concreto y define un intervalo temporal mediante las fechas de inicio y fin (**startDateTime** y **endDateTime**). El estado del bloqueo se gestiona mediante el enumerado **BlockStatus**, que permite indicar si el bloqueo está activo o ha sido cancelado.

Las relaciones entre estas entidades reflejan el funcionamiento del sistema. Un usuario puede tener múltiples reservas, mientras que cada reserva pertenece a un único usuario. Del mismo modo, un espacio puede tener múltiples reservas y múltiples bloqueos de mantenimiento, pero cada reserva o bloqueo está asociado a un único espacio.

Además, el sistema aplica ciertas reglas de negocio relacionadas con la disponibilidad de los espacios. En particular, no se permite crear reservas que se solapen temporalmente con otras reservas activas o con bloqueos de mantenimiento activos sobre el mismo espacio.



4 Aspectos técnicos y de diseño relevantes

La aplicación ha sido desarrollada siguiendo una arquitectura basada en capas, utilizando las buenas prácticas recomendadas en el desarrollo con Spring Boot.

La capa de **controladores** se encarga de gestionar las peticiones HTTP provenientes de la interfaz de usuario. Cada controlador está especializado en un conjunto de funcionalidades concretas, como la gestión de usuarios, espacios o reservas.



La capa de **servicios** contiene la lógica de negocio de la aplicación. En esta capa se implementan las reglas del sistema, como la validación de reservas, la comprobación de solapes entre reservas o bloqueos, y la gestión de la disponibilidad de los espacios.

La capa de **repositorios** se encarga del acceso a la base de datos mediante el uso de Spring Data JPA. Esta capa permite realizar operaciones de persistencia sobre las entidades del sistema de forma sencilla y desacoplada de la lógica de negocio.

En cuanto a la seguridad, se ha utilizado **Spring Security** para gestionar la autenticación y autorización de los usuarios. De esta manera, se garantiza que las funcionalidades de administración solo puedan ser utilizadas por usuarios con rol de administrador y que los usuarios estándar solo puedan acceder a sus propios recursos.

También se han implementado diferentes mecanismos de validación para asegurar la integridad de los datos, como la validación de contraseñas seguras, la comprobación de fechas válidas en las reservas y la prevención de solapamientos entre reservas y bloqueos de mantenimiento.

5 Descripción de funcionalidades desarrolladas por cada miembro del equipo

Una tabla que indique que funcionalidades desarrolladas cada uno de los integrantes del equipo (a grandes rasgos esto debería coincidir con el repositorio de GitHub). Por cada miembro del equipo, se deberá incluir un pantallazo de los commits de GitHub (filtrado por cada alumno).

Miembro del equipo	Funcionalidades desarrolladas
Omayma El Imami Gaada	Omayma se encargó principalmente de todo lo relacionado con la gestión del login de usuarios dentro de la aplicación. En concreto, desarrolló los siguientes requisitos junto con sus correspondientes pruebas: 1. Registro de usuario estándar en el sistema, incluyendo sus pruebas asociadas (<i>Pruebas 1–4</i>). 2. Inicio de sesión para usuarios registrados y administradores, junto con sus pruebas (<i>Pruebas 5–8</i>). 3. Cierre de sesión para usuarios registrados y administradores, así como sus pruebas correspondientes (<i>Pruebas 9–10</i>). 4. Modificación de contraseña para usuarios estándar, con sus respectivas pruebas (<i>Pruebas 37–38</i>).



	5. Seguridad del sistema (protección de recursos y acciones), incluyendo sus pruebas (<i>Pruebas 40–41</i>). Además, tras la finalización del desarrollo principal, también se encargó de la elaboración de los siguientes diagramas del proyecto: 6. Diagrama de navegabilidad, junto con su correspondiente explicación. 7. Diagrama entidad–relación, acompañado de su explicación. Por otro lado, participó en la revisión de la internacionalización de la aplicación, corrigiendo errores que se estaban produciendo en este ámbito. Finalmente, también colaboró en la redacción del informe final en Word.
Ikram El Mabrouk Morhnane	Ikram se encargó principalmente de la parte relacionada con la gestión de espacios y su disponibilidad dentro del sistema, así como de la implementación de dos requisitos opcionales. En concreto, desarrolló los siguientes requisitos junto con sus correspondientes pruebas: • Requisito 9: Consultar el listado global de reservas (<i>Pruebas 23–25</i>). • Requisito 10: Consultar el listado de espacios disponibles (<i>Pruebas 26–27</i>). • Requisito 11: Consultar el detalle de un espacio (<i>Prueba 28</i>). • Requisito 12: Consultar la disponibilidad de un espacio (<i>Prueba 29</i>). Asimismo, también implementó los siguientes requisitos opcionales: • Requisito 21: Reservas recurrentes (<i>Pruebas 42–43</i>). • Requisito 22: Límite de reservas activas por usuario (<i>Pruebas 44–45</i>). Además, contribuyó resolviendo diversos <i>warnings</i> detectados durante el desarrollo y participó en la redacción del documento final (Word) y se encargó de añadir una funcionalidad adicional relacionada con el perfil de usuario.



Moisés Montes Iglesias	Del mismo modo, Moisés se encargó principalmente de la parte relacionada con la gestión de administradores y el control de accesos según los distintos roles de usuario. En este contexto, desarrolló los siguientes requisitos junto con sus correspondientes pruebas: 1. Registrar nuevos espacios en el sistema, junto con sus pruebas (<i>Pruebas 11–14</i>). 2. Editar espacios existentes, con sus pruebas correspondientes (<i>Pruebas 15–16</i>). 3. Activar/Desactivar espacios (alta/baja lógica), junto con sus pruebas (<i>Pruebas 17–18</i>). 4. Crear bloqueos de mantenimiento para un espacio, incluyendo sus pruebas (<i>Pruebas 19–21</i>). 5. Cancelar bloqueos de mantenimiento, junto con su prueba correspondiente (<i>Prueba 22</i>). 6. Consultar el listado global de reservas, con sus pruebas asociadas (<i>Pruebas 23–25</i>). Además, participó en la resolución de errores tanto en el código como en algunos tests, especialmente aquellos derivados de la integración de los distintos commits del equipo y en la revisión de la internacionalización de la aplicación. También se encargó de la documentación del código mediante comentarios en formato Javadoc en los controladores (Controllers). Finalmente, colaboró en la redacción del documento final (Word).
Mario de la Mora Martínez	De igual forma, Mario se encargó principalmente de la funcionalidad relacionada con la gestión de reservas por parte del usuario estándar, incluyendo su creación, consulta y cancelación. En concreto, desarrolló los siguientes requisitos junto con sus correspondientes pruebas: 1. Registrar una nueva reserva, junto con sus pruebas (<i>Pruebas 30–31</i>). 2. Evitar solapes de reservas, con sus pruebas correspondientes (<i>Pruebas 32–33</i>).



	3. Consultar el listado de reservas propias, junto con sus pruebas (<i>Pruebas 34–35</i>). 4. Cancelar una reserva propia, con su prueba asociada (<i>Prueba 36</i>). Además, también implementó la internacionalización de vistas, junto con sus pruebas correspondientes (<i>Prueba 39</i>). Por otro lado, desarrolló el siguiente requisito opcional: • Requisito 23 (Administrador): Exportación de reservas a CSV, junto con sus pruebas (<i>Prueba 46</i>). Finalmente, participó en la redacción del documento final (Word).
--	--

★ Commits de Omayma El Imami Gaada:



Commits on Mar 18, 2026

Merge pull request #19 from MoisesMontesIglesias/uo300838-arreglo-warnings-y-otros-problemas **Verified** dca770b <>

arreglo de warnings y otros problemas del proyecto 20042ae <>

Commits on Mar 17, 2026

Merge pull request #14 from MoisesMontesIglesias/chores/datainitializer-warnings-ajustes **Verified** e538f5a <>

arreglo de tests y modificación de contraseña 61a4817 <>

Merge pull request #12 from MoisesMontesIglesias/moises **Verified** 16a2b5d <>

Commits on Mar 16, 2026

Merge pull request #10 from MoisesMontesIglesias/test-funcionalidades-23-45 **Verified** 3c4a08 <>

Commits on Mar 15, 2026

arreglo de tests e implementación del apartado de seguridad fba275d <>

Commits on Mar 14, 2026

implementacion cambio de contraseña y arreglo de algunos test 8086da3 <>

Commits on Mar 9, 2026

Merge pull request #1 from MoisesMontesIglesias/login-signup-signout-uo300838 **Verified** acdf79e <>

Commits on Mar 1, 2026

Implementación de autenticación (login, registro y logout). 51a4563 <>

Activar Windows
Ve a Configuración para activar Windows.

★ Commits de Ikram El Mabrouk Morhnane:

Commits

master

Ikram-em All time

Commits on Mar 18, 2026

Fin de corrección de todo a9c0583 <>

Merge pull request #18 from MoisesMontesIglesias/moises **Verified** 826fa3a <>

Merge pull request #17 from MoisesMontesIglesias/correccion-pruebas-spaceseleniumfunctionaltest **Verified** 17ddc49 <>

test: correccion pruebas SpacesSeleniumFunctionalTest 261a7a6 <>

Merge pull request #16 from MoisesMontesIglesias/fix/selenium-last-tests-stability **Verified** ace587b <>

fix: estabilizar ultimas pruebas selenium y paginacion global 97d0fe8 <>

Commits on Mar 17, 2026



Merge pull request #16 from MoisesMontesIglesias/fix/selenium-last-tests-stability			Verified	ace587b	<>
ikram-em authored 14 hours ago					
fix: estabilizar ultimas pruebas selenium y paginacion global				97d0fe8	<>
ikram-em committed 14 hours ago					
Commits on Mar 17, 2026					
Ajustes: correcciones en DataInitializer, warnings y otros ajustes				70e939f	<>
ikram-em committed 19 hours ago					
Merge pull request #13 from MoisesMontesIglesias/uo300838-revision-errores			Verified	b5f3401	<>
ikram-em authored yesterday					
Merge pull request #5 from MoisesMontesIglesias/moises			Verified	1289f28	<>
ikram-em authored yesterday					
Commits on Mar 16, 2026					
Add Selenium tests for requirements 23-29 and 42-45				c09c90b	<>
ikram-em committed 2 days ago					
Merge pull request #9 from MoisesMontesIglesias/opcional-21-22-ojo			Verified	aa5adfc	<>
ikram-em authored 3 days ago					
Implementa reservas opcionales y visibilidad de contraseña				57121d1	<>
ikram-em committed 3 days ago					
Merge pull request #8 from MoisesMontesIglesias/creado-homepage			Verified	a47a14d	<>
ikram-em authored 3 days ago					
Merge pull request #7 from MoisesMontesIglesias/seguridad-y-tests-uo300838			Verified	ab299bb	<>
ikram-em authored 3 days ago					
Creado homepage				50ac30a	<>
ikram-em committed 3 days ago					
Commits on Mar 9, 2026					
Merge pull request #2 from MoisesMontesIglesias/creada-parte-del-listado-de-espacios-y-disponibilidad-de-las-reservas			Verified	e3bbc9f	<>
ikram-em authored last week					
Implementa listado y disponibilidad de espacios con pruebas funcionales				37a1aa2	<>
ikram-em committed last week					

★ Commits de Moisés Montes Iglesias:

Commits on Mar 18, 2026		
Corregir errores finales sobre la vista, tamaño de botones, algunas partes que faltaban por internacionalizar y corregir que todos los tests funcionen. En resumen, errores menores		
MoisesMontesIglesias committed 2 hours ago		
Merge pull request #15 from MoisesMontesIglesias/moises		
MoisesMontesIglesias authored 15 hours ago		
Arreglo de errores en internacionalización		
MoisesMontesIglesias committed 15 hours ago		
Commits on Mar 17, 2026		
Solución de todos los errores en los tests. Falta pulir algunas partes y está terminado		
MoisesMontesIglesias committed yesterday		
Merge remote-tracking branch 'origin/moises' into moises		
MoisesMontesIglesias committed yesterday		
Arreglo de todos los conflictos con los commits de los diferentes participantes del grupo.		
MoisesMontesIglesias committed yesterday		
Arreglo de todos los conflictos con los commits de los diferentes participantes del grupo.		
MoisesMontesIglesias committed yesterday		
Pruebas casi terminadas de todos los requisitos		
MoisesMontesIglesias committed 2 days ago		



This screenshot shows a list of GitHub commits for the user MoisesMontesglesias, organized by date. The commits include:

- Committed on Mar 15, 2026:**
 - Comprobada la pull request** (Verified, d8a2dbf)
 - Requisito 9. Consultar el listado global de reservas.** (9edef6a)
 - Requisito 8. Cancelar bloques de mantenimiento.** (d72fee6)
 - Requisito 7. Crear bloques de mantenimiento para un espacio.** (d0a528f)
- Committed on Mar 14, 2026:**
 - Requisito 6. Activar/desactivar espacios (alta/baja lógica).** (ec06b50)
- Committed on Mar 13, 2026:**
 - Requisito 5. Editar espacios existentes.** (f6936ed)
- Committed on Mar 12, 2026:**
 - Merge de mis commits** (Verified, 39d94f3)
- Committed on Mar 11, 2026:**
 - Merge remote-tracking branch 'origin/moises' into moises** (781746f)
 - Ejercicio 4. Registrar nuevos espacios en el sistema. Completado al 100% e integrado en la interfaz. Tuve que modificar la url para que me permita ver la web** (1205285)
 - Ejercicio 4. Registrar nuevos espacios en el sistema. Completado al 100% e integrado en la interfaz. Tuve que modificar la url para que me permita ver la web** (c152e14)

★ Commits de Mario de la Mora Martínez:

This screenshot shows a list of GitHub commits for the user uo281860 (Mario de la Mora Martínez), organized by date. The commits include:

- Committed on Mar 16, 2026:**
 - Merge pull request #11 from MoisesMontesglesias/Mario** (Verified, uo281860)
 - Finalizacion exportacion CSV** (uo281860)
- Committed on Mar 15, 2026:**
 - Merge pull request #6 from MoisesMontesglesias/Mario** (Verified, uo281860)
 - Finalizacion requisitos y pruebas reservas** (uo281860)
 - Merge branch 'master' of https://github.com/MoisesMontesglesias/sdi2526-entrega1-n into Mario** (uo281860)
- Committed on Mar 13, 2026:**
 - Finalizacion accines. inicio pruebas selenium** (uo281860)
 - Commi internalizacion** (uo281860)
- Committed on Mar 12, 2026:**
 - Commit rama mario** (uo281860)
 - Merge branch 'master' of https://github.com/MoisesMontesglesias/sdi2526-entrega1-n into Mario** (uo281860)
 - Commit rama mario** (uo281860)

6 Conclusión individual de cada miembro del grupo.

→ Conclusión de Omayma El Imami Gaada:

El trabajo me gustó especialmente por el equipo; nos entendimos muy bien desde el principio y se creó un buen ambiente de colaboración que facilitó mucho el desarrollo de la práctica. Al principio el proyecto me daba un poco de miedo, ya que nunca había trabajado con Spring Boot, pero a medida que avanzamos



me di cuenta de que muchas de las estructuras y conceptos eran similares a lo que habíamos visto en clase, lo que me permitió afianzar conocimientos que antes no tenía tan claros.

Spring Boot resultó bastante cómodo de usar, y el manejo de Git también fue muy útil, ya que lo habíamos practicado antes y nos permitió coordinar los cambios sin problemas.

Una de las principales dificultades fue dividir el trabajo, ya que algunas funcionalidades dependían de otras y, en ocasiones, había que esperar a que otro compañero completara su parte antes de poder continuar. Aun así, gracias a la buena comunicación y a la organización del equipo, conseguimos coordinar nuestras tareas y avanzar de manera ordenada, lo que nos permitió cumplir con los plazos establecidos y mantener la coherencia del proyecto.

En conclusión, ha sido una experiencia muy positiva: aprendí a trabajar en equipo, reforcé mis conocimientos de Spring Boot y Git, y entendí la importancia de la coordinación y la comunicación para sacar adelante un proyecto completo.

→ Conclusión de Ikram El Mabrouk Morhnane:

Este trabajo me ha servido para mejorar tanto a nivel técnico como en el trabajo en equipo. La forma en la que estaban planteados los requisitos ha ayudado bastante, ya que estaban bien organizados y permitían ir avanzando poco a poco, aunque en algunos casos había que pensar bien cómo encajar unas funcionalidades con otras.

La parte que desarrollé, relacionada con los espacios y su disponibilidad, junto con los requisitos opcionales, me resultó interesante porque implicaba tener en cuenta bastantes condiciones y comprobar que todo funcionara correctamente.

Spring Boot me ha parecido bastante cómodo de usar, sobre todo por cómo organiza el proyecto en controladores, servicios y repositorios, lo que facilita mucho el desarrollo. También el uso de Git ha sido clave para poder trabajar en equipo y juntar todas las partes sin demasiados problemas.

Las pruebas con Selenium han sido probablemente lo más delicado, ya que cualquier pequeño cambio podía hacer que fallasen, pero aun así han sido muy útiles para comprobar que todo funcionaba como debería.

En cuanto al trabajo en grupo, creo que ha sido una buena experiencia, ya que nos hemos organizado bien y hemos podido cumplir con los plazos sin grandes problemas. En general, ha sido un proyecto bastante completo que me ha ayudado a aprender y mejorar en varios aspectos.

→ Conclusión de Moisés Montes Iglesias:

El diseño de cómo está estructurado lo que se nos pide para el proyecto, nos hace muy cómodo el trabajo ya que tiene la información relevante muy bien descrita y nos permite un grado de libertad en como implementarlo y mostrarlo en la web, algo que es de agradecer.

En cuanto a la implementación, las similitudes con el proyecto de Spring desarrollado durante las prácticas han hecho que tener la base del proyecto no haya requerido mucho problema, no como ha sido la implementación de algunas especificaciones, como en mi caso que me han llevado más tiempo los bloqueos de mantenimiento y como representarlos, pero nada fuera de lo imposible.



Las pruebas con Selenium han sido la parte más frágil porque cualquier cambio pequeño en la interfaz puede romperlas, pero son muy útiles al simular el uso real de la aplicación.

SpringBoot me parece un muy buen framework de trabajo que me ha sorprendido para bien, ya que no soy muy fan del desarrollo web, y sin embargo, este proyecto me ha mantenido enganchado durante el proceso de desarrollo haciendo que quiera mejorar todos los detalles, por lo que estoy muy contento con el trabajo realizado tanto en clase como en casa de este framework.

En cuanto al trabajo en grupo, por suerte, me ha tocado un grupo trabajador y que ha cumplido con los tiempos, por lo que mi experiencia con el trabajo en grupo ha sido gratificante y creo que es una buena práctica para reducirnos la carga de trabajo, reforzar nuestro conocimiento y trabajo en git y resolver adversidades como los conflictos al mezclar nuestras partes desarrolladas de forma individual.

En resumen, destaco la integración en equipo, el uso de git, Spring Boot como framework y las pruebas con Selenium.

→ Conclusión de Mario de la Mora Martínez:

Este trabajo me ha servido para consolidar la habilidad de trabajar en equipo, como siempre la comunicación es clave ya que al mínimo cambio realizado pueden surgir conflictos los cuales pueden llevar a una gran pérdida de tiempo. Consolidé también mi conocimiento del uso de git para el control de versiones del proyecto.

Sobre Spring Boot sólo tengo que decir que me ha parecido muy cómodo para la organización del proyecto: controladores, servicios y repositorios que encajan perfectamente y facilita añadir funcionalidad nueva. Maven ha sido clave para la gestión de dependencias.

Las pruebas con Selenium han sido un poco lo más frágil por decirlo de alguna forma ya que al mínimo cambio que no coincida en un texto o cualquier cosa ya se rompen, aun así me han parecido muy útiles ya que simulan el uso real sin que tengas que probarlo tú mismo.

En resumen, a destacar, la integración de cambios con el equipo, uso de git, uso de SpringBoot como framework y el uso de pruebas con Selenium.

7 Información necesaria para el despliegue y ejecución

La aplicación ha sido desarrollada utilizando las herramientas y tecnologías empleadas durante las clases de la asignatura.

Las principales tecnologías utilizadas son:

- **Java:** versión 17
- **Spring Boot:** framework principal para el desarrollo de la aplicación web
- **Spring Security:** gestión de autenticación y autorización
- **Spring Data JPA:** acceso y gestión de la base de datos
- **HSQLDB:** base de datos utilizada para almacenar la información del sistema
- **JUnit 5:** desarrollo de pruebas unitarias
- **Selenium:** pruebas funcionales automatizadas

Para ejecutar la aplicación es necesario importar el proyecto en un entorno de desarrollo compatible con proyectos **Maven**, como IntelliJ IDEA. Una vez importado el proyecto, la aplicación puede ejecutarse mediante la clase principal de Spring Boot.



Al iniciar la aplicación, la base de datos se inicializa automáticamente con datos de prueba, incluyendo un usuario administrador, varios usuarios estándar, espacios disponibles, reservas y bloqueos de mantenimiento, lo que permite ejecutar las pruebas automatizadas sin necesidad de configuración adicional.

8 Conclusión

El trabajo en equipo ha sido clave para hacer la práctica, ya que ha sido necesario coordinar las diferentes partes del desarrollo y mantener una comunicación constante entre los miembros del grupo. Esto ha permitido distribuir las tareas y avanzar de forma continua en la implementación de los requisitos.

Durante el desarrollo del proyecto también nos encontramos con algunas dificultades. Al principio no teníamos muy claro cómo empezar a organizar la aplicación ni cómo estructurar las diferentes partes del sistema. Poco a poco, a medida que fuimos avanzando y probando distintas soluciones, conseguimos entender mejor cómo debían relacionarse los distintos componentes del proyecto y basándonos en los proyectos de clase.

En conclusión, la realización de esta práctica ha supuesto una experiencia muy útil para tener más conocimientos sobre el desarrollo de aplicaciones web con Spring, el trabajo colaborativo mediante repositorios de control de versiones y la importancia de las pruebas automatizadas para garantizar la calidad del software.